Final Report: Methods to Improve the Sensitivity of Wheat Genomic Prediction Models

Calvin Cao

Western University

Scholar's Electives 2200E

Professor Mike Domaratzki

April 4th, 2025

Final Report: Methods to Improve the Sensitivity of Wheat Genomic Prediction Models

As the world's population grows and climate change renders more farmland unarable, food resources will be stretched ever thinner. One crop of concern is wheat, which accounts for up to 1 in 4 calories consumed worldwide (D'Odorico and Carr, 2014) yet is one of the least climate-resilient major food crops, with 15-27% of all wheat farmland to be rendered unarable in the case of 2–3-degree global warming (Heikonen et al., 2025).

Plant breeders constantly seek ways to improve crop yields while minimizing land use and have created genetically modified variants with higher yields per acre than naturally occurring species. However, there are endless possible gene modifications and limited resources to plant and evaluate their performance, with the process of developing a single new crop variant taking up to 10 years and millions of dollars (Brennan and Martin, 2005). One promising approach to accelerate discovery is genomic prediction (GP). GP relies on machine learning models that learn from phenotypic and genotypic data collected from planting trials. They can then predict the performance of theoretical genetically modified lines based solely on gene sequences and physically plant only the most promising ones (Montesinos-López et al., 2023, pp. 2-3). Because researchers aim to significantly reduce the number of lines that require physical testing, training data typically contains many more 'non-top' (poor-performing) than 'top' (high-performing) lines. As a result of these imbalanced datasets, however, GP models often have worse sensitivity than specificity, i.e., they are more accurate in classifying non-top lines than top lines (Fernandez et al., 2018, pp. 1-2). Yet missing a top line with exceptional yield is often costlier than planting and testing a few extra non-top lines, meaning more balanced models are preferred. Montesinos-López et al. (2023) investigated several approaches to improving sensitivity-specificity balance in GP models. In this project, three strategies for further improving the sensitivity of their models were investigated: classification threshold tuning, non-linear ML algorithms, and linear interpolation data augmentation.

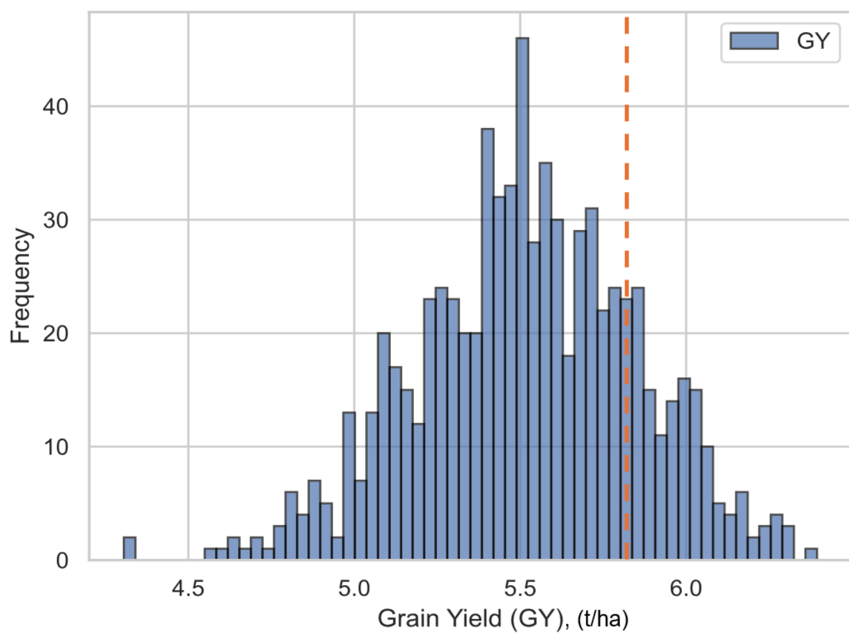
Methods

Dataset

The same dataset used by Montesinos-López et al., EYT_1, was used. It is from Elite Yield Trials conducted by the International Maize and Wheat Improvement Center (CIMMYT, 2018) and contains the tonne per hectare harvest of 766 wheat variants (y) across 4 different growing environments and a 766 by 766 genetic relationship matrix (X), which is a simplification of each line's genetic sequence where each column X_j represents the similarity of sample $X^{(j)}$ to each of the 766 other samples.

Figure 1.0

Histogram of Unscaled Grain Yield of Wheat Lines, ETY_1



Note. Orange dotted line represents original top line classification threshold used by Montesinos-Lopez et al., representing the 0.8^{th} quantile of all samples.

Original Training Pipeline and Modifications Made

Montesinos-Lopez et al. compared three training pipelines named R, B, and RO, which reformulated genomic prediction as a classification problem and/or optimized the post-training top line classification threshold. See *Part 1: Non-linear ML Algorithms and Threshold Tuning* for a detailed description of each training process. They ultimately found post-training threshold optimization without reformulation (model RO) to produce the most accurate models.

All their models used the GBLUP ML algorithm, which is a linear mixed model:

$$y = X\beta + Zu + \varepsilon$$

In this case, y is the grain yield of each wheat line, $X\beta$ represents fixed effects like the planting environment, Zu is the random genetic effects following a normal distribution with covariance determined by the genomic relationship matrix, and ε is the residual error. GBLUP's strength is that it makes full use of pairwise genetic relationships the GRM encodes by using it as a covariance matrix, accounting for shared ancestry and relatedness between samples. However, it also assumes each gene has an independent, additive linear contribution to the phenotype output. In reality, non-linear genetic interactions known as *epistasis* can occur (Montesinos-López et al., 2023). In this project, Models R, B, and RO will be modified to use Support Vector Machines (SVM) and XGBoost. SVM maps data to a higher dimensional space and separates classes using a hyperplane and is known to perform well with high-dimensional datasets like EYT_1. Extreme Gradient Boosting (XGB) can similarly model complex gene interactions using decision tree ensembles (Uddin et al., 2019, p. 12). It is hypothesized that these non-linear machine learning models will better capture complex gene interactions that distinguish top lines than GBLUP, thereby improving sensitivity.

Another area for improvement is classification thresholds. Aside from the post-training classification thresholds optimized using ROC curves in models B and RO, all Montesinos-Lopez's models use separate arbitrarily chosen classification thresholds at other steps during training. For example, for model B they reformulate the grain yield regression model as a classification model by converting the target variable to a binary variable, assigning "top line" or "not top line" labels to each with a cutoff at the 0.8^{th} quantile of the dataset. In this project, the three models will be trained using more balanced values for these arbitrarily chosen classification thresholds, which may improve specificity while ensuring acceptable sensitivity.

Finally, Montesinos-Lopez et al. (2024) tried using the *mixup* data augmentation technique developed by Zhang et al. (2018) to generate synthetic top line samples to reduce class imbalance. *mixup* uses linear interpolation between random pairs of training examples. They found that it unexpectedly decreased overall classification accuracy, suggesting the augmentation simply introduced noise. One potential reason for this is that the average distance between samples in datasets generally increases as dimensionality increases, since there are more dimensions between which two points can be separated. The further away two points are in a dataset, the lower the chance that an interpolated point between them will be representative of the true distribution. The datasets tested by Montesinos-Lopez had almost 800 features. Thus, it is hypothesized that augmenting the top lines of the imbalanced GP datasets using more conservative algorithms that do not interpolate across vast distances will lead to improved performance on top lines.

Part 1: Non-linear ML Algorithms and Threshold Tuning

Montesinos-Lopez et al. (2023) trained all three models (Models R, B, and RO) using GBLUP as the ML algorithm and the dataset's 0.8^{th} quantile for all untuned classification thresholds. (The continuous grain yield values were converted into binary labels at different points during the training process for each pipeline, meaning Model R, B and RO each used distinct untuned classification thresholds, τ_{pre}, τ_{post} and τ_{interm}, respectively. See below for specific definitions of each). In this project, versions of Models R, B, and RO were trained with every combination of non-linear ML Algorithm (XGB, SVM) and untuned threshold $(0.5, 0.6, 0.7, 0.8, 0.9)$.

Model R

1. Data Preparation
   1.1. The original dataset contained 4 grain yield values for each line across four different environments. To simplify it, they were averaged to produce one grain yield per line.

1.2. Each column of the relationship matrix X and the grain yield $\overline{y}$ were standardized to have a mean of 0 and standard deviation of 1.

2. Model Training and Hyperparameter Tuning

   2.1. 5-fold outer-fold cross-validation was conducted, with each split consisting of 20% test data and 80% training data.

      2.1.1. For each outer fold, a 10-fold inner cross-validation was performed, with each split consisting of 10% test data and 90% training data.

         2.1.1.1. A grid of regression models (SVM or XGBoost) with every possible set of hyperparameters in Table 1.0 and Table 1.1 was trained.

         2.1.1.2. Predictions on the inner CV test set were made and the Pearson coefficient was calculated for each model (see Figure 2.0 for an example). Pearson Correlation Coefficient measures the linear relationship between two variables (-1 to 1). Higher absolute values indicate stronger correlation.

      2.1.2. The mean hyperparameters of the 5 models with the best Pearson coefficients across all inner folds were calculated. Using all training data of the outer fold, a final model was trained using these average hyperparameters.

      2.1.3. Continuous tonne per hectare grain yield predictions ($\hat{y}$) using this final model were made on the outer fold's test set (see Figure 2.1 for example results).

      2.1.4. These predictions were classified as top (1) or not top (0). First, a classification threshold t_{post} was set at the value at the quantile τ_{post} of $\overline{y}$. (τ_{post} is the threshold for which the values $\{0.5, 0.6 \dots 0.9\}$ were explored.) t_{post} was then used to classify both $\overline{y}$ and $\hat{y}$ to create $\overline{y}_{bin}$ and $\hat{y}_{bin}$.

3. For each outer fold, the average of the 10 inner-fold best models' classification metrics (F1, sensitivity, specificity, and kappa) were calculated and reported using $\overline{y}_{bin}$ and $\hat{y}_{bin}$. F1 Score is the harmonic mean of precision and recall (0 to 1). Higher values indicate better balance

between false positives and false negatives. Sensitivity is the proportion of actual positives correctly identified. Higher values indicate fewer false negatives. Specificity is the proportion of actual negatives correctly identified. Higher values indicate fewer false positives. Cohen's Kappa measures agreement between predicted and actual classifications, adjusting for chance (−1 to 1). Higher values indicate better reliability.

4. These 5 outer fold averages were themselves finally averaged and reported (Table 2.0).

Figure 2.0

Example of Inner-CV Grid Search: Predicted vs. Actual Values and Pearson Coefficients for Models of Every Parameter Combination (Gamma vs. Regularization Parameter), Model R, SVM, $\tau_{post} = 0.5$*, Outer Fold 0, Inner Fold 0*

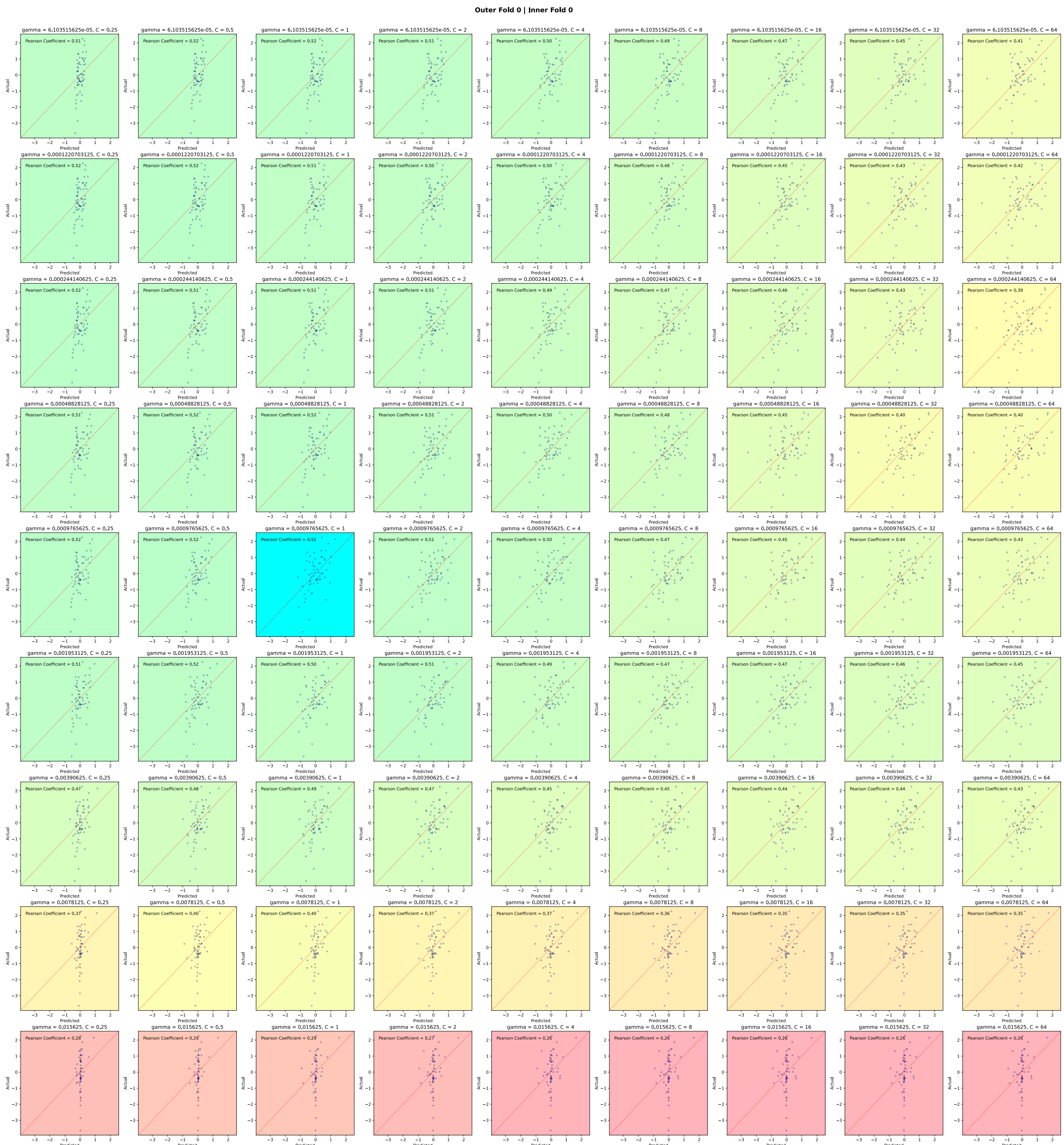
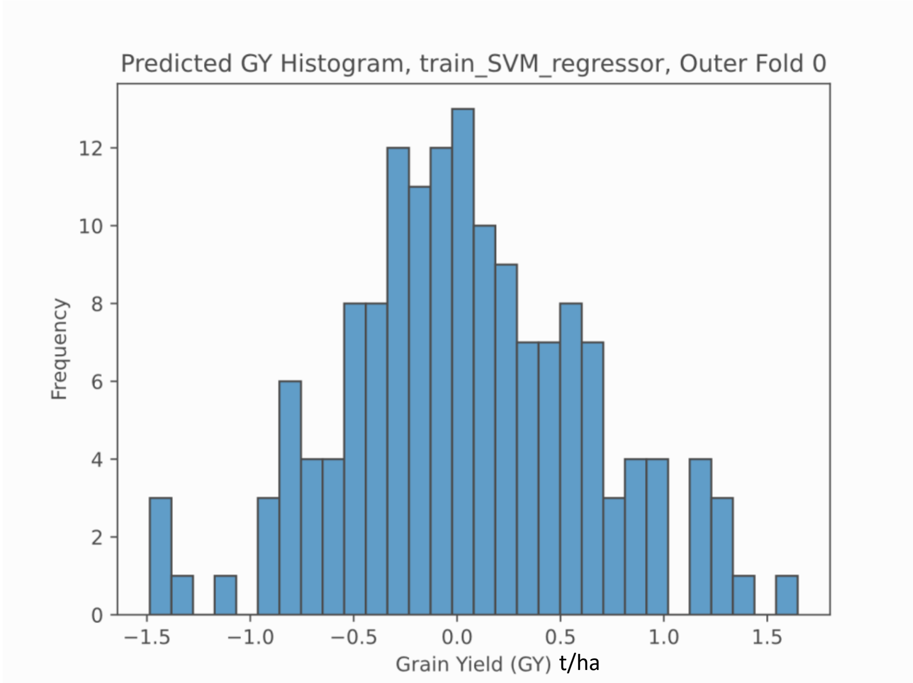


Figure 2.1

Histogram of Predicted Transformed GY for Outer-Fold Model, Model R, SVM, $\tau_{post} = 0.5$*,*

Outer Fold 0, Inner Fold 0



Model B

1. Data Preparation

   1.1 Grain yield values were averaged as with Model R.

   1.2 The continuous training labels $\bar{y}$ were converted to a binary vector where $\bar{y}_{bin_i} = 1$ if $\bar{y}_i$ exceeded the quantile threshold τ_{pre}, and 0 otherwise. (This is the threshold for which values of $\{0.5,\ 0.6 \ldots 0.9\}$ were explored.) X was then transformed to have a mean of 0 and standard deviation of 1.

2. Model Training and Hyperparameter Tuning

   2.1 5-fold outer-fold cross-validation was conducted, with each split consisting of 20% test data and 80% training data.

      2.1.1 For each outer fold, a 10-fold inner cross-validation was performed, with each split consisting of 10% test data and 90% training data.

         2.1.1.1 Classification models with every possible set of hyperparameters were trained. Note that $\bar{y}_{bin}$ and not $\bar{y}$ was used as the training labels.

2.1.1.2 Intermediate top line probability predictions, $\hat{y}_{interm_{prob}}$, on the inner CV test set were made for each model. $\hat{y}_{interm_{bin}}$ was then calculated by applying a simple 0.5 quantile binning threshold to $\hat{y}_{interm_{prob}}$. Finally, F1 scores were calculated using $\hat{y}_{interm_{bin}}$ and $\overline{y}_{bin}$ (see Figure 3.0 for example grid search).

2.1.1.3 The model with the best F1 score was chosen and its parameters were saved.

2.1.1.4 Using $\hat{y}_{interm_{prob}}$ of the best model and $\overline{y}_{bin}$, the optimal classification threshold, t_{post} (a t/ha grain yield value, not a quantile), to minimize mean squared difference between sensitivity and specificity was chosen.

2.1.2 Using all training data of the outer fold, a final model was trained using the mean best hyperparameters of the 10 inner folds.

2.1.3 Categorical predictions using this final model, $\hat{y}_{final_{bin}}$, were made on the outer fold's test set by classifying the final model's probability predictions using the average t_{post} of the 10 inner folds.

3. The average classification metrics (F1 Score, Sensitivity, Specificity, Kappa) between the $\hat{y}_{final_{bin}}$ of each outer fold and $\overline{y}_{bin}$ was calculated and reported (Table 2.1).

Figure 3.0

Example of Inner-CV Grid Search: ROC Curves and F1 Scores for Models of Every Parameter Combination (Gamma and Regularization Parameter), Model B, SVM, $\tau_{post} = 0.5$, Outer Fold

0, Inner Fold 0

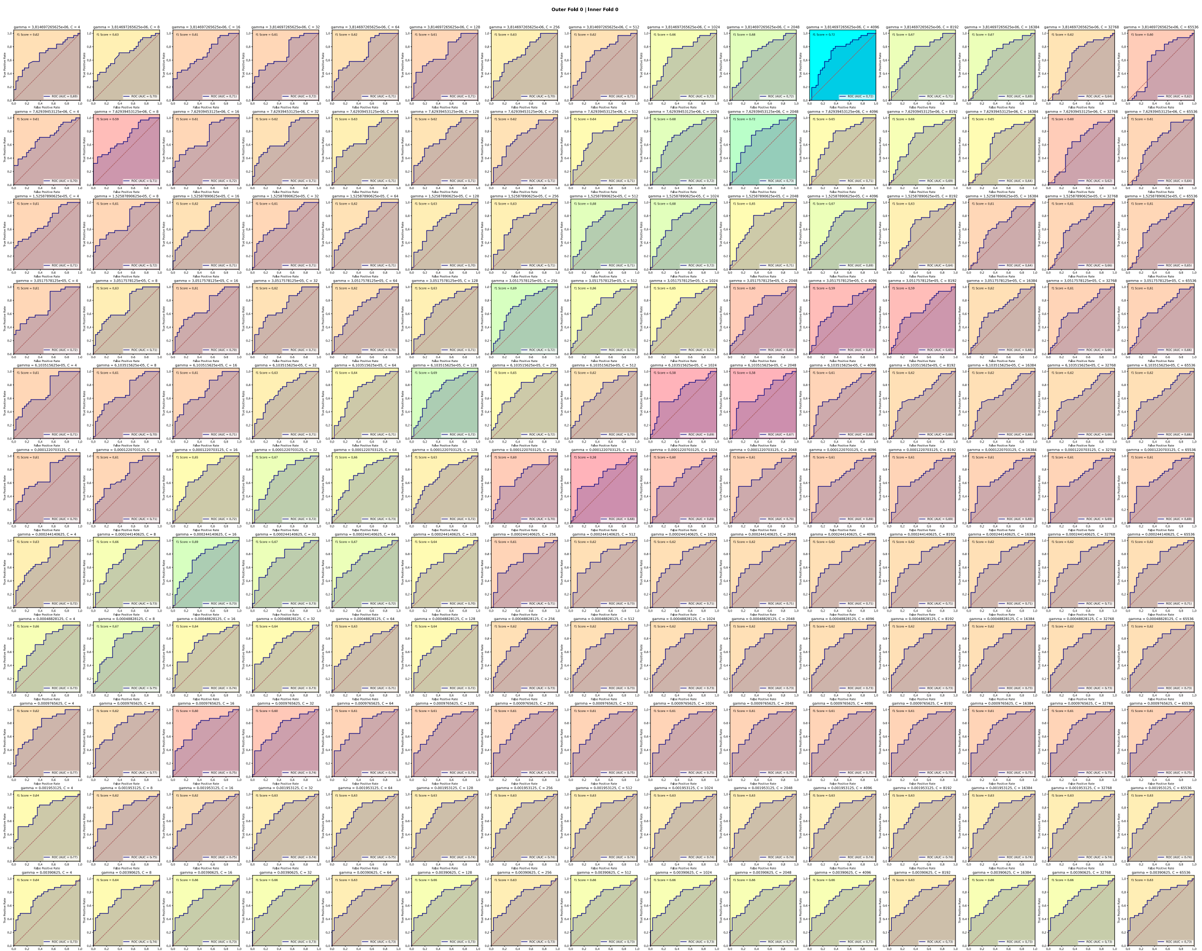
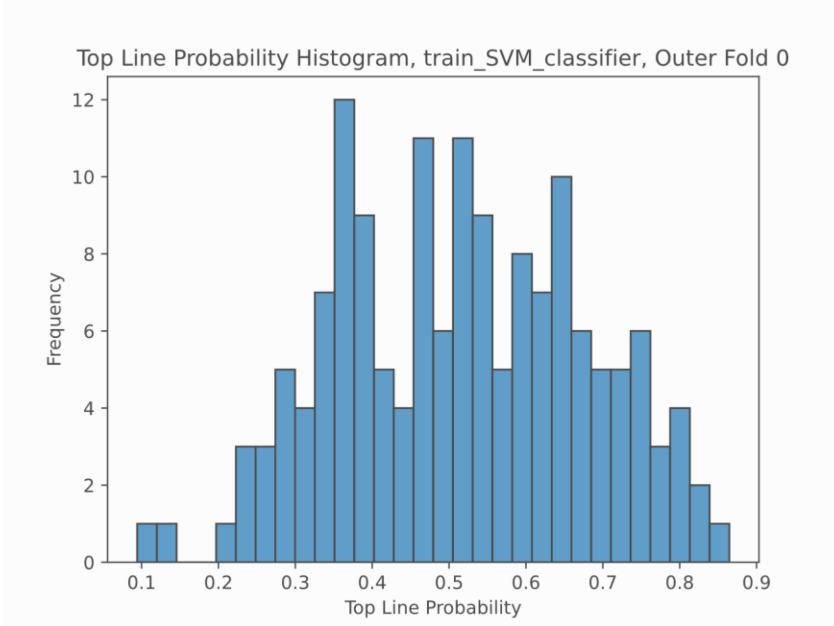


Figure 3.1

Example of Outer-CV Grid Search: Histogram of Predicted Top Line Probability for Outer-Fold Model, Model B, SVM, $\tau_{post} = 0.5$, Outer Fold 0, Inner Fold 0



Model RO

Model RO is identical to Model R except that after the model with the optimal hyperparameters is selected during inner-fold CV, a threshold optimization step similar to the one in Model B was implemented. Using an arbitrary "intermediate" quantile threshold τ_{interm}, for which values $\{0.5, 0.6\ \ldots\ 0.9\}$ were explored, $\overline{y}$ was converted into the binary vector $\overline{y}_{bin}$. Then, the optimal fixed-value threshold (not quantile) t_{post} that minimized the mean squared difference between sensitivity and specificity between the predictions and $\overline{y}_{bin}$ was calculated. The average of each inner fold's t_{post} was then used to classify the predictions of the final outer CV model. Note that $\overline{y}$ was used to train the models while $\overline{y}_{bin}$ was used solely during the threshold optimization step.

Table 1.0

Range of Parameters Explored during Each Inner-Fold CV for SVM

GS Approach	Gamma (γ)	Regularization Parameter
Model R	$\{2^{-14}, 2^{-13} \ldots 2^{-6}\}$	$\{2^{-2}, 2^{-1} \ldots 2^{6}\}$
Model B	$\{2^{-18}, 2^{-17} \ldots 2^{-8}\}$	$\{2^{2}, 2^{3} \ldots 2^{16}\}$

Model RO	$\{2^{-14}, 2^{-13} \dots 2^{-6}\}$	$\{2^{-2}, 2^{-1} \dots 2^{6}\}$

Note. The range of values to explore was determined through a prior coarse grid search, following the best practices outlined by Hsu et al. (2016).

Table 1.1

Range of Parameters Explored during Inner-Fold CV for XGBoost

GS Approach	Number of Estimators	Maximum Depth
Model R	$\{13, 25, 50, 100, 200\}$	$\{1,2,3,4,6,10,16\}$
Model B	$\{3,7,13,25,50,100,200,400\}$	$\{1,2,3,4,6,10,16,32,64\}$
Model RO	$\{3,7,13,25,50,100,200\}$	$\{1,2,3,4,6,10,16,32,64\}$

Note. The range of values to explore was determined through a prior coarse grid search following the best practices outlined by Hsu et al. (2016).

Part 2: Data Augmentation

In the second part of the project, two data augmentation methods more conservative than the *mixup* method used by Montesinos-Lopez were explored.

Model B with SMOTE

The Model B training process was further modified by adding SMOTE oversampling and undersampling prior to model training in each inner CV fold (before step 2.1.1.1). To study how different oversampling and undersampling intensities affect model performance, two distinct training groups were defined. The first group was trained with $\tau_{post} = 0.8$, $p_{over} = 1$, and every combination of ML Model (chosen from SVM or XGBoost) and p_{under} (chosen from $\{0.3, 0.55, 0.8, 1\}$). The second group was trained with $\tau_{post} = 0.8$, $p_{under} = 1$, and every combination of ML Model (chosen from SVM or XGBoost) and p_{over} (chosen from $\{0.3, 0.55, 0.8, 1\}$).

The SMOTE augmentation algorithm used is as follows:

1. Select minority class instance x_{m} randomly from the set of all top grain yield lines in the dataset.
2. Identify the 5 nearest neighbors of x_{m} by Euclidean distance. Randomly select one of these neighbors, x_{n}, and generate a new synthetic sample using linear interpolation:

$$x_{\mathrm{syn}} = x_m + \lambda \cdot (x_{\mathrm{n}} - x_m)$$

   where λ is a random number between 0 and 1 drawn from a beta distribution.
3. Repeat steps 1 and 2 until the total synthetic samples equals $p_{over}(1.5n)$, where n is the original number of non-synthetic minority class samples and p_{over} is a portion between 0 and 1 of these generated samples to keep. ($1.5n$ is the default number of synthetic samples generated by Kunz (2020)'s library used in the *Model RO and SMOGN* part of the project, so the total synthetic samples was defined this way for ease of comparison).
4. Randomly undersample the majority class by removing a portion of the original 'not top' samples until only p_{under} remains.

Model RO with SMOGN

The Model RO training process was further modified by adding SMOGN oversampling and undersampling prior to model training in each inner CV fold. Two groups of these augmented models were trained. The first group was trained with $\tau_{interm} = 0.8$, $p_{over} = 1$, and every combination of ML Model (chosen from SVM or XGBoost) and p_{under} (chosen from $\{0.3, 0.55, 0.8, 1\}$). The second group was trained with $\tau_{interm} = 0.8$, $p_{under} = 1$, and every combination of ML Model (chosen from SVM or XGBoost) and p_{over} (chosen from $\{0.3, 0.55, 0.8, 1\}$).

1. Classify all samples as either top lines or not top lines using τ_{interm}. (This is only for the purpose of data augmentation and the rest of the pipeline remains the same; the model trained is still a regression model.)

2. Repeat $n \times p_{\text{over}} \times 1.5$ times:

   2.1. Select a random sample x_m whose grain yield exceeds the majority class threshold and a random one of its 5 nearest neighbors x_n.

   2.2. x_n is marked "safe" if its distance from x_m , the seed instance, is below half of the median distance between x_m and its 5 nearest neighbours. Using the median ensures robustness against outliers, allowing interpolation only with genuinely close neighbors.

   2.3. If x_n is safe, linearly interpolate between the two to generate a new sample:

$$x_{\text{synth}} = \lambda x_m + (1 - \lambda) x_n$$

   λ is drawn from a uniform distribution.

   2.4. If the neighbour is not safe, add a small amount of gaussian noise to each feature to generate a new sample. $\sigma = 1$ was used, matching the standardized feature variance. This filtering prevents interpolation between dissimilar samples, which could lead to unrealistic synthetic data.

$$x_{synth} = x_m + N(0, \sigma^2)$$

3. Random undersampling: Randomly remove not top samples until only p_{under} of the original majority class samples remain.

Results

Figure 4.0

Non-Augmented Model R Classification Metrics by ML Algorithm and Post-Training Classification Threshold

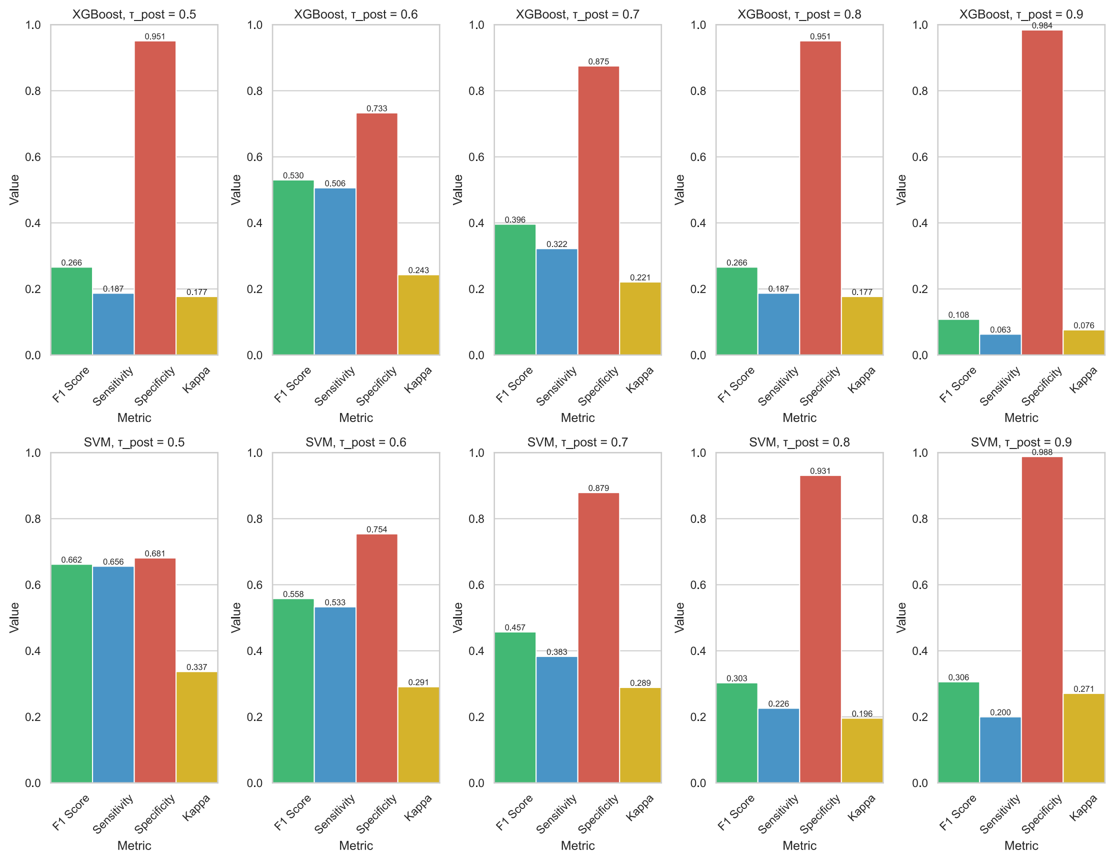


Figure 5.0

Non-Augmented Model B Classification Metrics by ML Algorithm and Pre-Training Classification Threshold

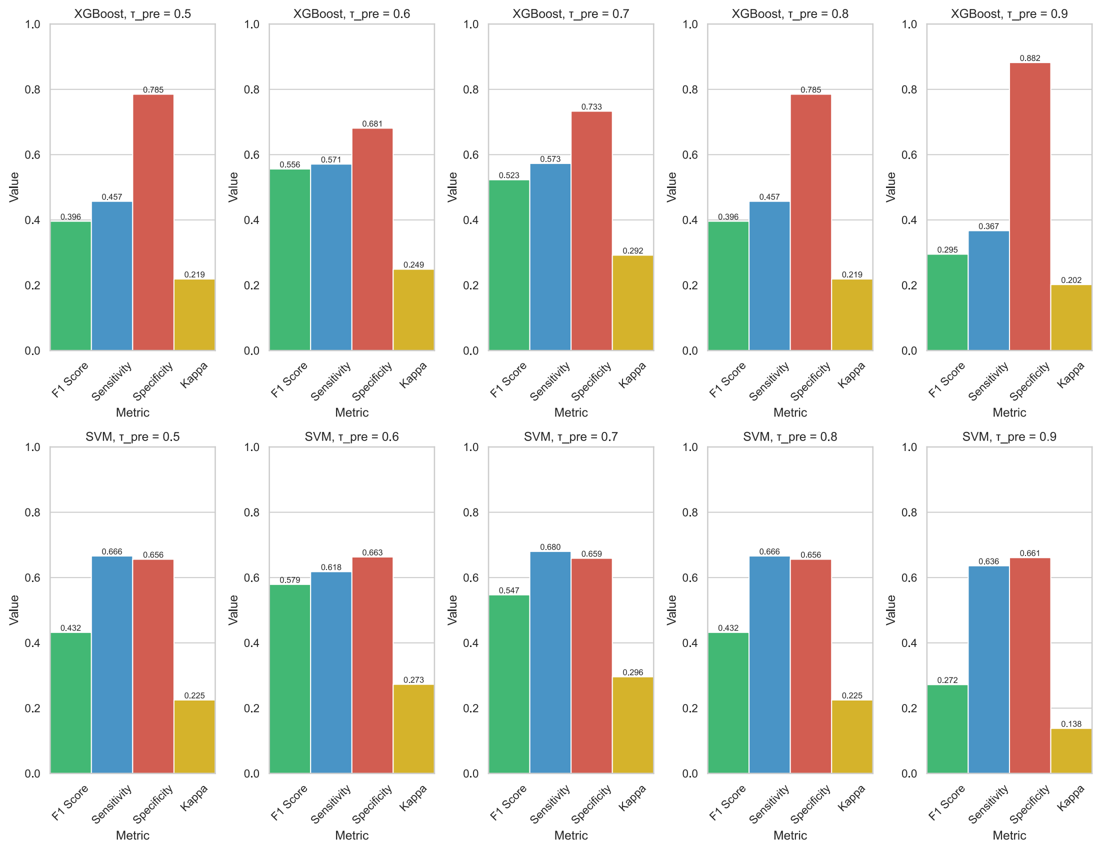


Figure 6.0

Non-Augmented Model RO Classification Metrics by ML Algorithm and Interim Training Classification Threshold

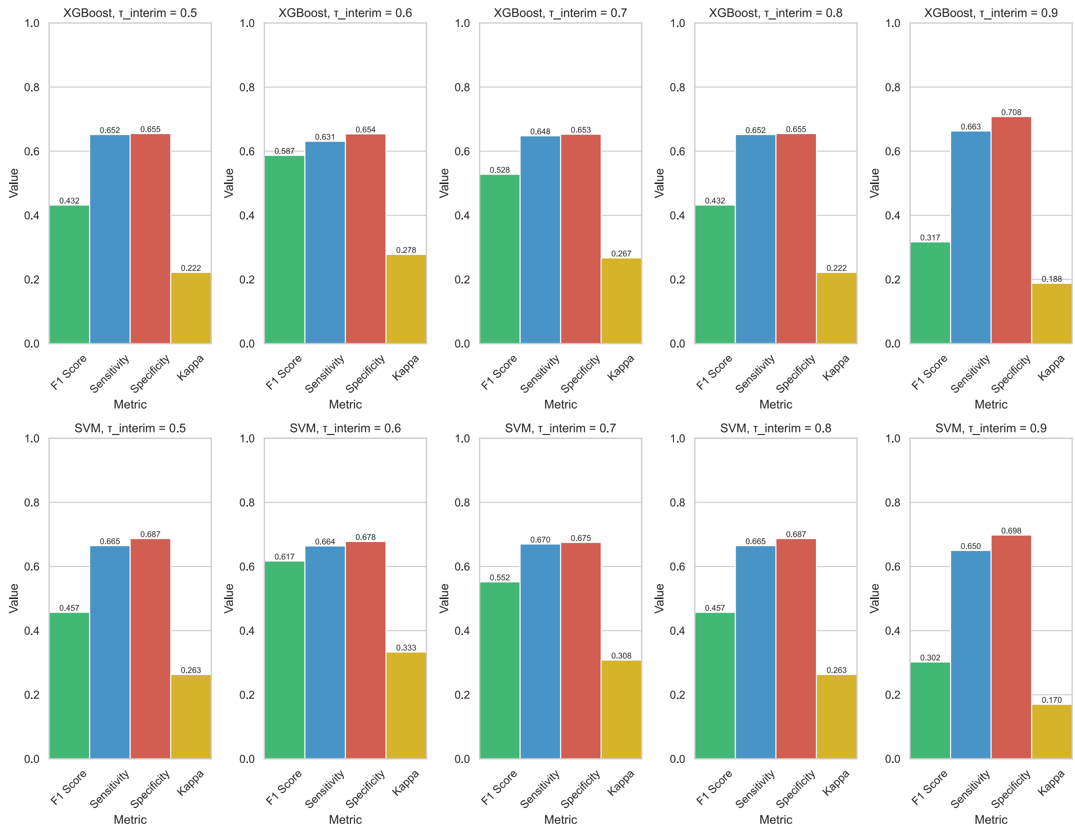

XGBoost, τ_interim = 0.5
0.432
0.652
0.655
0.222
XGBoost, τ_interim = 0.6
0.587
0.631
0.654
0.278
XGBoost, τ_interim = 0.7
0.528
0.648
0.653
0.267
XGBoost, τ_interim = 0.8
0.432
0.652
0.655
0.222
XGBoost, τ_interim = 0.9
0.317
0.663
0.708
0.188
SVM, τ_interim = 0.5
0.457
0.665
0.687
0.263
SVM, τ_interim = 0.6
0.617
0.664
0.678
0.333
SVM, τ_interim = 0.7
0.552
0.670
0.675
0.308
SVM, τ_interim = 0.8
0.457
0.665
0.687
0.263
SVM, τ_interim = 0.9
0.302
0.650
0.698
0.170
Value
Metric
F1 Score
Sensitivity
Specificity
Kappa

Figure 7.0

Classification Metrics, Augmented SVM Models, Synthetic Oversampling Rate = 1.0

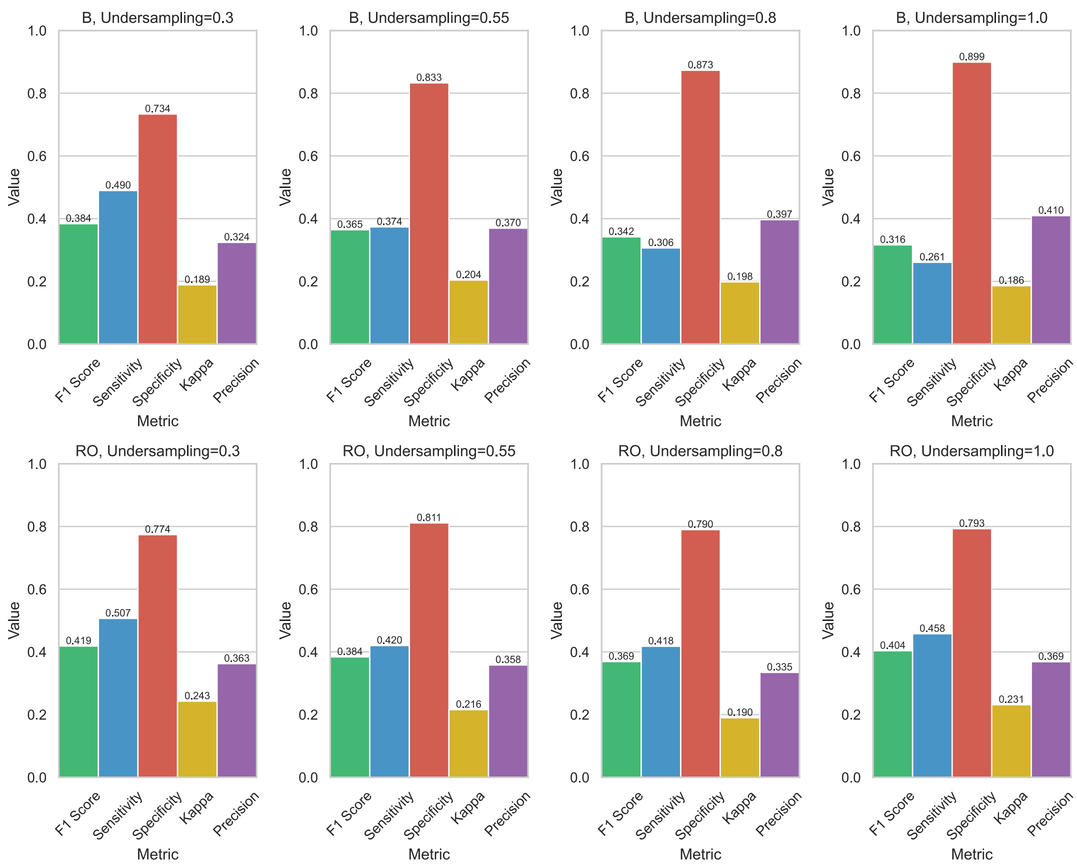


Figure 8.0

Classification Metrics, Augmented XGB Models, Synthetic Oversampling Rate = 1.0

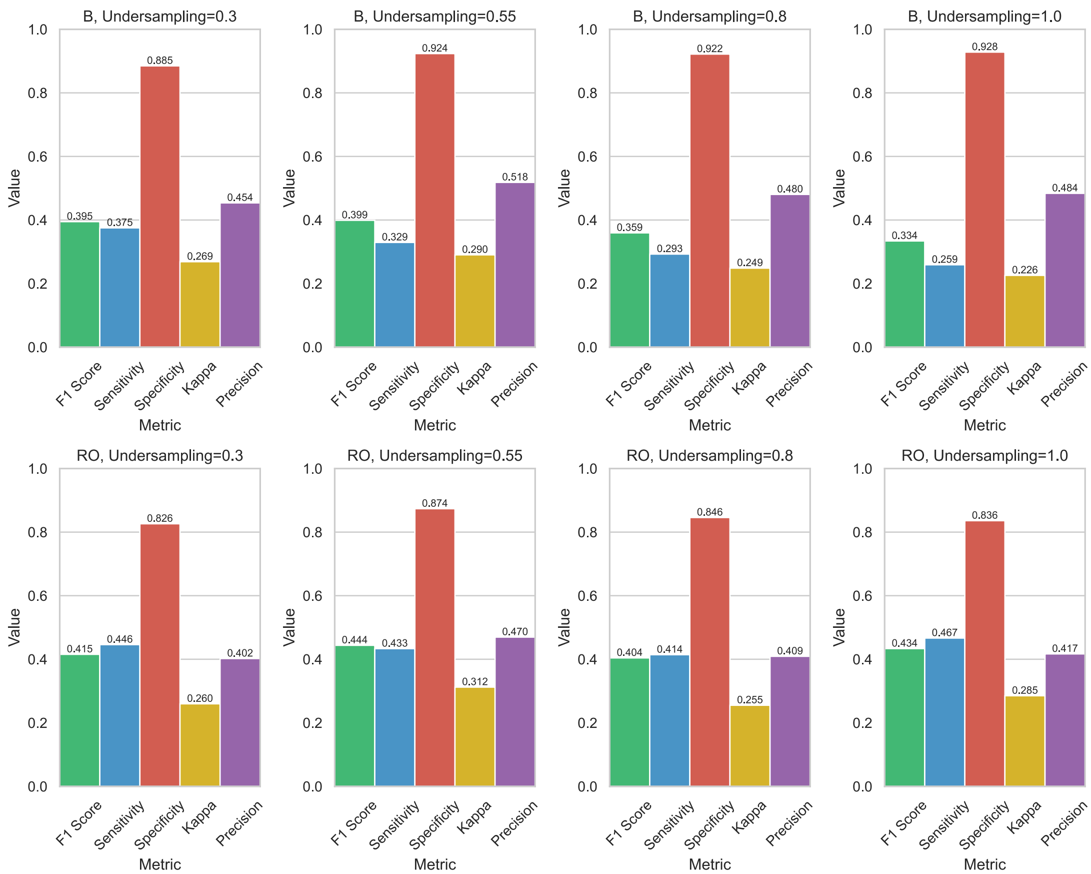


Figure 9.0

Classification Metrics, Augmented SVM Models, Majority Class Undersampling Rate = 1.0

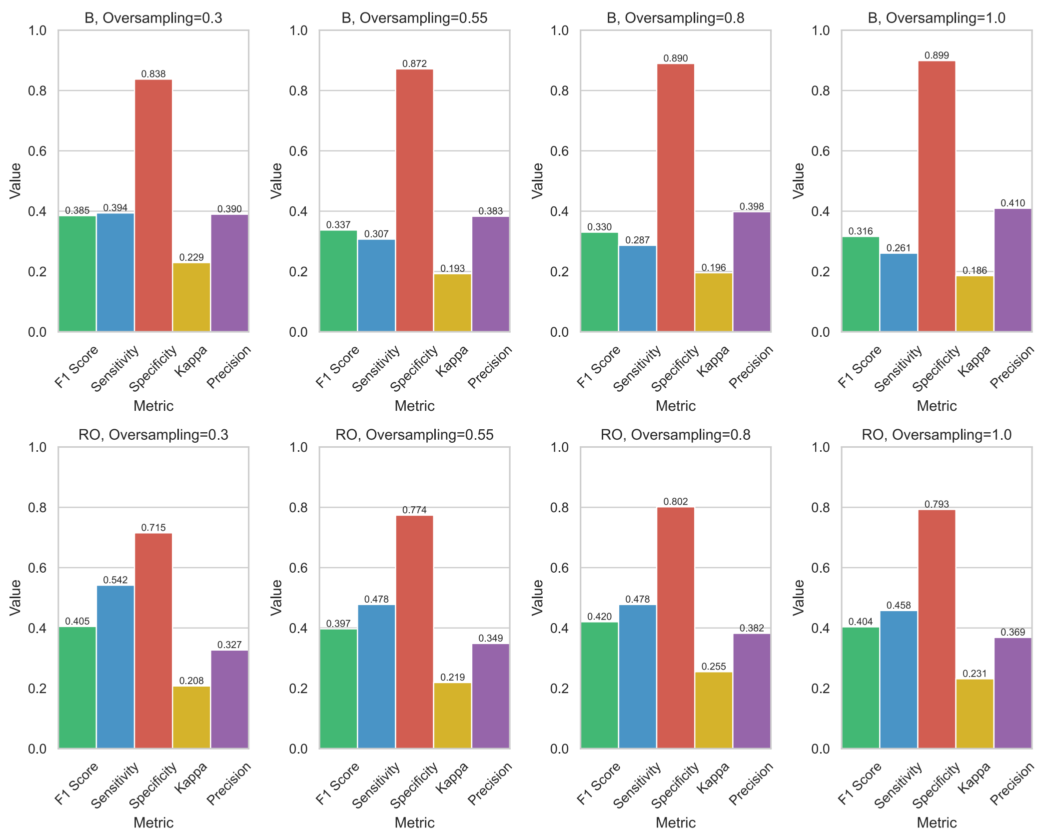


Figure 10.0

Classification Metrics, Augmented XGB Models, Majority Class Undersampling Rate = 1.0

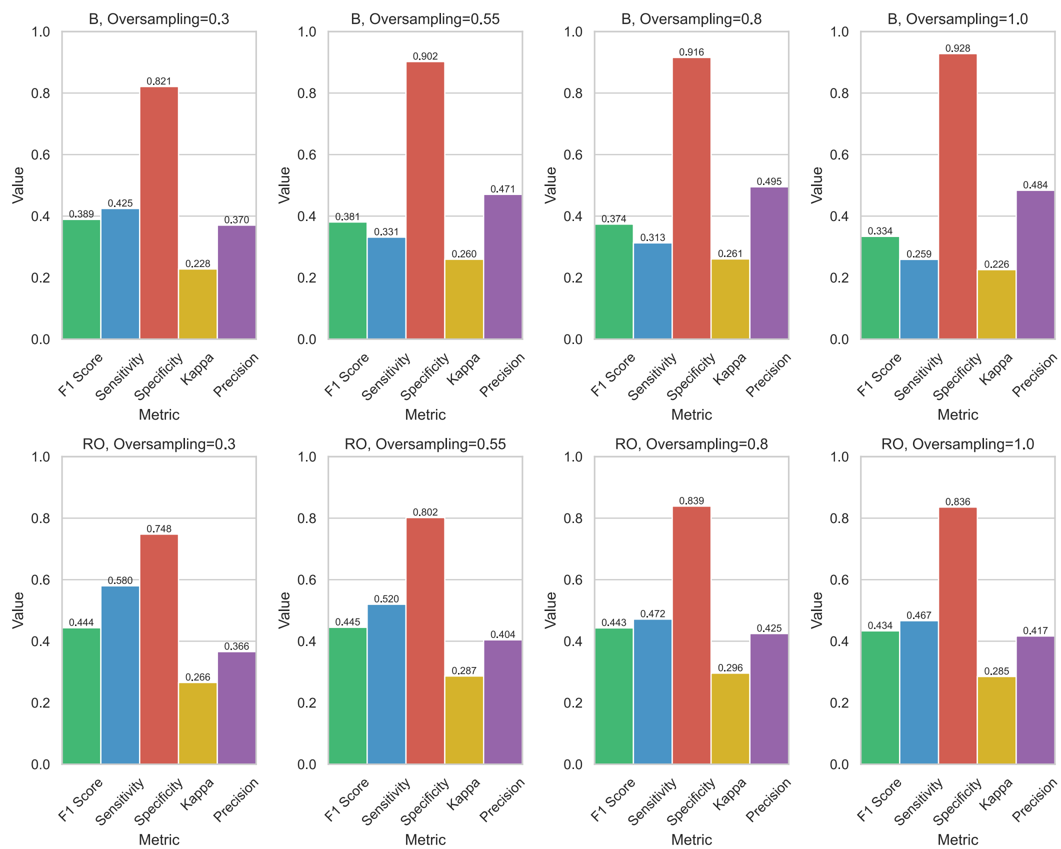
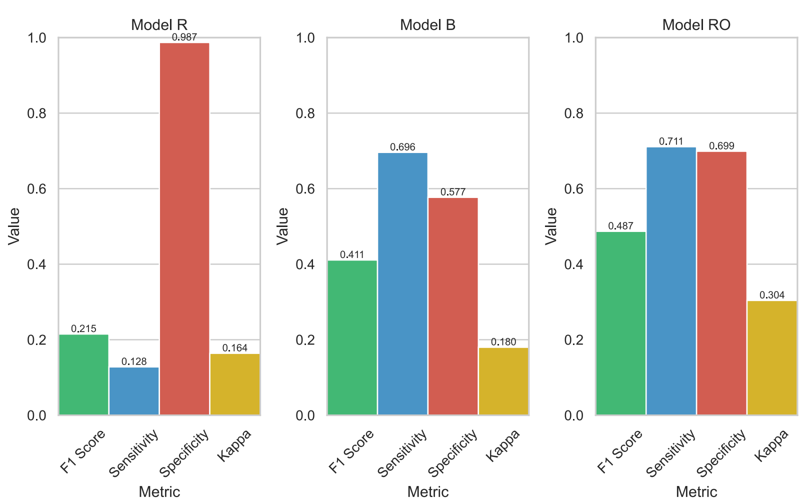


Figure 11.0

GBLUP Models R, RO, and B Accuracy Metrics (Montesinos-Lopez et al., 2023)



Note: All arbitrary classification thresholds for the three models were set to the 0.8^{th} quantile of the entire training set.

Discussion

As expected, models trained using lower top line classification thresholds generally produced a better balance between sensitivity and specificity (e.g. the τ_{post} = 0.5 SVM Model R had F1 = 0.662, substantially better than τ_{post} = 0.9 SVM Model R which had F1 = 0.306), since lower thresholds for τ_{pre}, τ_{post} and τ_{interm} all favor models that perform better on more balanced datasets. While these simple changes effectively just "lower the bar" for what is considered a top line rather than improving the predictive accuracy of the model generally, they could be valuable tweaks for breeders with resources to test more candidates to avoid accidentally eliminating top lines early in the breeding process. Future research related to the classification threshold could explore the impact of threshold optimization based on precision and recall instead of sensitivity and specificity. Precision is the proportion of instances that the model classified as positive that are actually positive, while recall is the same as sensitivity. Optimizing for these metrics has been shown to improve performance on imbalanced datasets as they focus more on minority class performance.

Comparing the ML algorithms, SVM models had better sensitivity and F1 scores than XGBoost models across almost all thresholds for Models R and B. SVM and XGBoost were similar across most thresholds when using Model RO, though the best ML algorithm/threshold combination for RO between SVM and XGB still belonged to SVM with $\tau_{interm} = 0.6$, which had F1 $= 0.587$ and sensitivity $= 0.631$. Thus, overall, SVM was more effective than XGBoost, likely due to the high dimensionality of the dataset ($d = 766$); as mentioned earlier, SVM accuracy suffers less than XGBoost with high-dimensional datasets. The $\tau_{pre} = 0.8$ SVM and XGB Model R algorithms both had superior F1 scores and sensitivity than the original 0.8 GBLUP Model R, however, the original 0.8 GBLUP B and RO models generally performed better than 0.8 SVM and XGBoost B and RO models, suggesting that while SVMs and XGB models work better when no post-training threshold optimization is applied, the ceiling for GBLUP's accuracy after threshold tuning may be higher than that of SVM or XGBoost.

This signifies that GBLUP is best at assigning the highest grain yield predictions to the actual top performers but is too conservative in the magnitude of these top predictions, resulting in them missing the top line classification threshold. When the classification threshold is properly tuned, GBLUP still outperforms the non-linear models. There are several potential reasons for this. SVM and XGBoost models, unlike GBLUP, essentially flatten the genomic relationship matrix and interpret each column as an isolated feature. The loss of this relationship information may make it more difficult to model linear polygenic interactions. This seems to have outweighed any accuracy gain from SVM or XGBoost's ability to model non-linear epistasis relationships.

The second part of the experiment found that as the portion of majority class kept after undersampling decreases towards 0.3 when using SMOTE with Model B, sensitivity increases, and specificity decreases. There is no consistent trend with model RO and SMOGN. Sensitivity decreases and specificity increases as synthetic oversampling proportion increases when using SMOTE with Model B with both XGB and SVMs. This trend generally also holds with RO and SMOGN.

Overall, in most cases, F1 scores were lower for the augmented data compared to models with the same parameters but with no data augmentation. Many factors could explain the persistent inefficacy of linear interpolation data augmentation despite the more conservative approach used compared to Montesinos-Lopez et al.'s *mixup*. Blagus and Lusa (2013) demonstrated that in high-dimensional spaces, SMOTE tends to amplify variance in the minority class while failing to capture the characteristics that define class boundaries. This can lead to synthetic samples that exaggerate noise in the original data rather than enhancing the signal relevant to classification tasks. Another issue specific to genetic datasets is that linear interpolation may struggle to capture epistasis interactions, producing biologically impossible synthetic samples rather than learning the nuance of, for example, one gene completely cancelling out the effect of another.

Conclusion

The adjustments to Montesinos-Lopez et al.'s training processes produced mixed results. Lowering the arbitrary thresholds used did result in a closer sensitivity-specificity balance. Additionally, using ML algorithms like SVM and XGBoost can improve the performance of Model R, the model without post-training threshold optimization.

However, Montesinos-López et al.'s GBLUP remains superior to SVM and XGBoost when post-training classification threshold tuning is applied, and linear interpolation data augmentation algorithms generally either did not change or worsened model sensitivity. Thus, to further improve imbalanced GP models, more sophisticated ML algorithms and data augmentation strategies capable of modelling inter-gene interactions either through explicit consideration of known biological pathways or more flexible universal approximation abilities may be necessary. For example, deep learning models like Generative Adversarial Networks have been shown by Montserrat et al. (2019) and Li et al. (2023) to create synthetic genomic data that better reflect realistic marker combinations; these work by pitting a generator neural network against a discriminator that attempts to distinguish the generator's synthetic data from true data. Further testing on crop-specific datasets could yield insights into the effectiveness of GANs in this domain. Alternatively, reducing dimensionality prior to linear interpolation data augmentation through sequential feature selection or regularization that incorporates known pathways may help improve signal clarity. For example, Linkage Disequilibrium Pruning could be employed, which calculates correlation coefficients based on genetic variant cooccurrence frequency and removes markers too highly correlated (Calus and Vandenplas, 2018). Even standard principal component analysis (PCA) has been shown to improve ML model performance on genetic datasets (Sun et al., 2024) and may also improve the effectiveness of SMOTE and SMOGN by partially remedying the curse of dimensionality.

References

Blagus, R., & Lusa, L. (2013). SMOTE for high-dimensional class-imbalanced data. *BMC Bioinformatics*, *14*(1), 106. https://doi.org/10.1186/1471-2105-14-106

Branco, P., Torgo, L., & Ribeiro, R. P. (n.d.). *SMOGN: a Pre-processing Approach for Imbalanced Regression*.

Brennan, J. P., & Martin, P. J. (n.d.). *Developing Cost Functions for a Wheat Breeding Program*.

Calus, M. P. L., & Vandenplas, J. (2018). SNPrune: An efficient algorithm to prune large SNP array and sequence datasets based on high linkage disequilibrium. *Genetics Selection Evolution*, *50*(1), 34. https://doi.org/10.1186/s12711-018-0404-z

CIMMYT. (2018). *CIMMYT Global Wheat Breeding Program EYT Dataset* [Dataset]. https://data.cimmyt.org/dataset.xhtml?persistentId=hdl:11529/10548140

D'Odorico, P., Carr, J. A., Laio, F., Ridolfi, L., & Vandoni, S. (2014). Feeding humanity through global food trade. *Earth's Future*, *2*(9), 458–469. https://doi.org/10.1002/2014EF000250

Fernandez, A., Garcia, S., Herrera, F., & Chawla, N. V. (2018). SMOTE for Learning from Imbalanced Data: Progress and Challenges, Marking the 15-year Anniversary. *Journal of Artificial Intelligence Research*, *61*, 863–905. https://doi.org/10.1613/jair.1.11192

Heikonen, S., Heino, M., Jalava, M., Siebert, S., Viviroli, D., & Kummu, M. (2025). Climate change threatens crop diversity at low latitudes. *Nature Food*. https://doi.org/10.1038/s43016-025-01135-w

Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (n.d.). *A Practical Guide to Support Vector Classification*.

Kunz, N. (2020). *Synthetic Minority Over-Sampling Technique for Regression with Gaussian Noise* (Version 0.1.2) [Python]. https://pypi.org/project/smogn/

Li, R., Wu, J., Li, G., Liu, J., Xuan, J., & Zhu, Q. (2023). Mdwgan-gp: Data augmentation for gene expression data based on multiple discriminator WGAN-GP. *BMC Bioinformatics*, *24*(1), 427. https://doi.org/10.1186/s12859-023-05558-9

Montesinos-López, O. A., Kismiantini, & Montesinos-López, A. (2023). Two simple methods to improve the accuracy of the genomic selection methodology. *BMC Genomics*, *24*(1), 220. https://doi.org/10.1186/s12864-023-09294-5

Montesinos-López, O. A., Solis-Camacho, M. A., Crespo-Herrera, L., Saint Pierre, C., Huerta Prado, G. I., Ramos-Pulido, S., Al-Nowibet, K., Fritsche-Neto, R., Gerard, G., Montesinos-López, A., & Crossa, J. (2024). Data Augmentation Enhances Plant-Genomic-Enabled Predictions. *Genes*, *15*(3), 286. https://doi.org/10.3390/genes15030286

Montserrat, D. M., Bustamante, C., & Ioannidis, A. (n.d.). *Class-Conditional VAE-GAN for Local-Ancestry Simulation*.

Siirtola, P., & Röning, J. (2020). Comparison of Regression and Classification Models for User-Independent and Personal Stress Detection. *Sensors*, *20*(16), 4402. https://doi.org/10.3390/s20164402

Sun, Y., Kong, L., Huang, J., Deng, H., Bian, X., Li, X., Cui, F., Dou, L., Cao, C., Zou, Q., & Zhang, Z. (2024). A comprehensive survey of dimensionality reduction and clustering methods for single-cell and spatial transcriptomics data. *Briefings in Functional Genomics*, *23*(6), 733–744. https://doi.org/10.1093/bfgp/elae023

Torgo, L., Ribeiro, R. P., Pfahringer, B., & Branco, P. (2013). SMOTE for Regression. In L. Correia, L. P. Reis, & J. Cascalho (Eds.), *Progress in Artificial Intelligence* (pp. 378–389). Springer Berlin Heidelberg.

Uddin, S., Khan, A., Hossain, M. E., & Moni, M. A. (2019). Comparing different supervised machine learning algorithms for disease prediction. *BMC Medical Informatics and Decision Making*, *19*(1), 281. https://doi.org/10.1186/s12911-019-1004-8

Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). *mixup: Beyond Empirical Risk Minimization* (No. arXiv:1710.09412). arXiv. https://doi.org/10.48550/arXiv.1710.09412